AI-Powered Project Topic Suggestion System

Content-based recommendation using GitHub data, a Knowledge Graph, TF-IDF and Cosine Similarity

Presented by: Manthena Naina | B.Tech CSE, ICFAI Tech
Guides: Dr. Sanjib Raul and Dr. Srinivasarao Dhanikonda

Problem Statement & Objectives

The problem

Students know their interests (e.g. "Python, NLP") but struggle to choose a project topic.

Manually browsing thousands of projects is slow, random and overwhelming.

Objectives

- Take student interests and skills as input
- Learn from real GitHub projects
- Add related ideas using a Knowledge Graph
- Rank and return Top-N personalized project topics
- Keep the model simple and fully explainable

Literature Review

Content-based recommendation

Recommends items by comparing their content with user interests; no cold-start problem.

Knowledge Graph integration

Entities and relations add semantic context beyond plain keywords.

Semantic similarity analysis

Measures how close two texts are, e.g. TF-IDF with cosine similarity.

Recommendation ranking

Scores are sorted so the most relevant items appear first (Top-N).

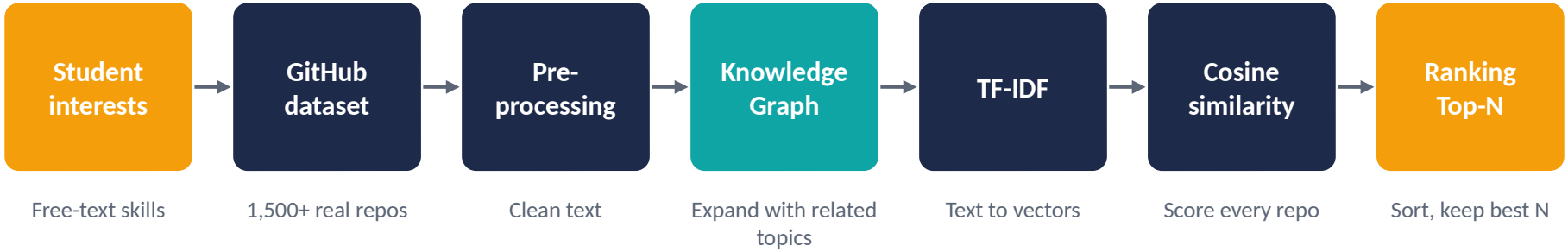
Paper (Author, Year)	Contribution	Relevance to this project
Salton & Buckley (1988), Information Processing & Management	Introduced TF-IDF term weighting for automatic text retrieval.	Basis of our TF-IDF vectors for repos and student queries.
Adomavicius & Tuzhilin (2005), IEEE TKDE	Surveys content-based, collaborative and hybrid recommenders and their limits.	Supports our content-based design with no cold-start problem.
Guo et al. (2022), IEEE TKDE	Surveys knowledge-graph recommenders for accuracy and explainability.	Backs our Knowledge Graph for expanding student interests.

Literature Review: Recent Work (2025-2026)

Paper (Author, Year, Venue)	Contribution	Relevance to this project
Pratomo & Utami (2026), AITI: Jurnal Teknologi Informasi	Content-based recommender for student final-project topics using TF-IDF and cosine similarity	Closest to our task; our baseline method
Kairbek et al. (2026), Computacion y Sistemas	Compares TF-IDF, BM25, MiniLM and DPR retrieval; BM25 gives best precision and ranking stability	Supports trying BM25 instead of cosine
Sumathi et al. (2025), ICSICE 2024 proceedings	GitHub API system that retrieves repositories by language, topic and popularity	Same data source as ours
Zhang et al. (2026), LNCS Service-Oriented Computing	ARrec: recommends GitHub awesome repositories using graph mining	Graph-based GitHub recommendation
Zangari et al. (2026), Machine Intelligence Research	Survey of transformer-based graph recommender systems	Shows how our simple Knowledge Graph can grow

Full citations are in the reference list of the research paper.

System Architecture



Output: a ranked list of recommended project topics with similarity scores

Dataset: Real GitHub Repositories

- Source: public GitHub Search API (most-starred repos across 20 topics)
- Size: 1,500+ unique repositories
- Fields: name, description, topics, language, stars
- Topics covered: machine learning, NLP, computer vision, web, IoT, blockchain, cybersecurity and more
- Stored as github_repos.csv and loaded with pandas; duplicates removed

Why real data?

Suggestions come from projects people actually built, so they are realistic and relevant.

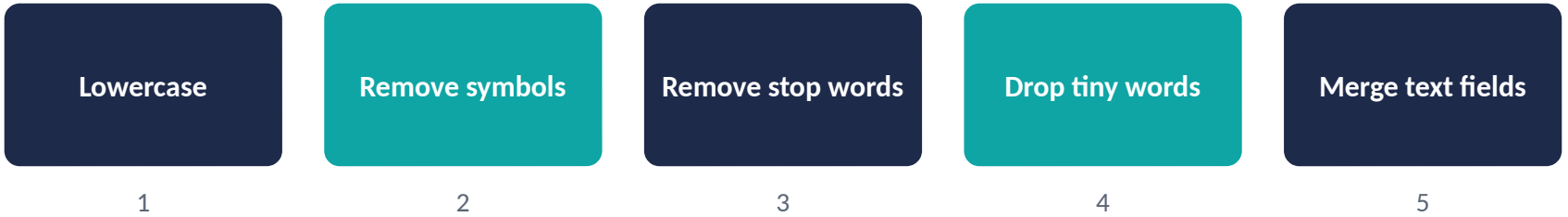
Database Connection (SQLite)

Database	Python library	Best for
SQLite (chosen)	**sqlite3 (built into Python)**	**Small and medium data; no server to install**
MySQL / PostgreSQL	mysql-connector-python / psycopg2	Multi-user systems; needs a running server
MongoDB	pymongo	JSON-style documents
Neo4j	neo4j driver	Graph data; a natural home for the Knowledge Graph

```
conn = sqlite3.connect("github_repos.db")
df.to_sql("repositories", conn, if_exists="replace", index=False)   # save
df = pd.read_sql("SELECT * FROM repositories", conn)                 # load
```

SQLite replaces github_repos.csv and allows SQL queries, e.g. WHERE language = 'Python' ORDER BY stars DESC.

Preprocessing

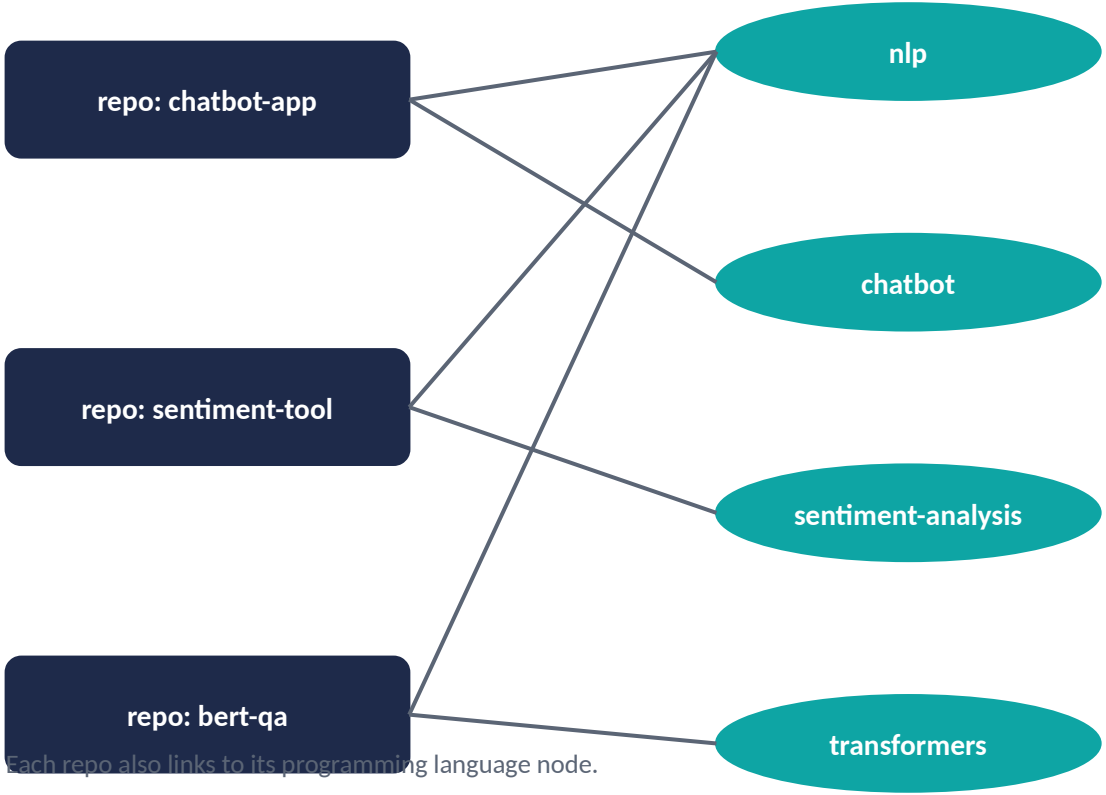


Example

Input: "Chatbot using Transformers for Question-Answering!"
Output: "chatbot transformers question answering"

Description + topics (twice) + language are joined into one document per repo.

Knowledge Graph (networkx)



Each repo also links to its programming language node.

How it is used

Student types "nlp".

Graph matches topic nlp, then adds topics that co-occur with it: chatbot, sentiment-analysis, transformers.

Result: related ideas the student did not mention.

TF-IDF: Turning Text into Numbers

TF

How often a word appears in this document.

IDF

log(total docs / docs containing the word). Rare words score high.

TF-IDF

TF x IDF. Frequent here but rare overall means a strong keyword.

- "project" appears everywhere, so its weight is near 0
- "transformers" appears in few repos, so its weight is high
- ngram_range=(1,2) keeps phrases like "machine learning"
- Every repo and the student query become vectors in the same space

Cosine Similarity

$$\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$$

1.0

Very similar (same direction)

0.5

Partly similar

0.0

No words in common

Example: Query [machine, learning, website] = [1, 1, 0]
Repo A "machine learning" = [1, 1, 0], score 1.0
Repo B "website design" = [0, 0, 1], score 0.0

Better Similarity: BM25 (vs. Cosine)

```
score(q,d) = SUM IDF(w) * f(w,d)(k1+1) / ( f(w,d) + k1(1 - b + b|d|/avgdl) )
```

IDF(w)

Rare words count more than common ones.

k1 = 1.5

Saturation: a word repeated 50 times does not score 50 times more.

b = 0.75

Length normalization: long descriptions are not favoured unfairly.

- A word mentioned 20 times counts about 2.3x one mention, not 20x
- Document length is handled by a tunable setting (b)
- Built for ranking a short query against documents; standard in search engines
- Usually stronger than TF-IDF cosine; confirm on our data with Precision@k

Similarity Measures: Limitations

Measure	How it works	Main limitations
Cosine (TF-IDF)	Angle between two vectors	Matches words, not meaning; needs vectors; no term saturation
Euclidean / Manhattan	Straight-line / summed difference	Affected by text length; poor in high dimensions
Jaccard	Shared items / all unique items	Ignores word weight and frequency; best for tags
Dice	Overlap (counted twice) / total size	Same limits as Jaccard: no weighting, no meaning
Pearson correlation	Linear relation of two vectors	Unstable on sparse text; better suited to ratings
BM25 (proposed)	**IDF + saturation + length normalization**	**Keyword-based (no synonyms); k1 and b need tuning**
Sentence-BERT + cosine	Compares meaning via embeddings	Needs a pretrained model; slower; harder to explain

Proposed: BM25. Next step: compare the methods with Precision@k.

Core Code: Recommendation Function

```
def recommend(df, G, interests, top_n=10):
    expanded = expand_with_knowledge_graph(G, interests)

    vectorizer = TfidfVectorizer(ngram_range=(1, 2))
    repo_matrix = vectorizer.fit_transform(df["doc"])
    query_vector = vectorizer.transform([clean(expanded)])

    scores = cosine_similarity(query_vector, repo_matrix).flatten()
    best = scores.argsort()[::-1][:top_n]

    result = df.iloc[best].copy()
    result["score"] = scores[best]
    return result[result["score"] > 0]
```

Execution & Output

```
python recommender.py

Enter your interests/skills: python, machine learning, nlp
Knowledge Graph matched topics : ['machine-learning', 'nlp']
Knowledge Graph added topics   : ['data-science', 'text-classification', 'recommender-system'] ...

TOP RECOMMENDED PROJECT TOPICS
1. f/spam-detect  (similarity: 0.514)
   Idea   : Spam email classifier with machine learning
   Topics : machine-learning, nlp
2. h/news-class   (similarity: 0.509)
   Idea   : News article classification
   Topics : nlp, machine-learning, text-classification
3. g/resume-parse (similarity: 0.465)
   Idea   : Resume parser using NLP
   Topics : nlp, machine-learning
```

Actual execution on a 16-repo sample dataset. Full-dataset (1,500+ repos) results will differ.

Output: Cosine Similarity (TF-IDF)

```
Enter your interests/skills: python, machine learning, nlp
Knowledge Graph matched topics : ['machine-learning', 'nlp', 'deep-learning', ...]
Knowledge Graph added topics   : ['transformers', 'computer-vision', 'data-science', ...]

TOP RECOMMENDED PROJECT TOPICS (COSINE)
1. h/news-class    (cosine score: 1.000)
   Idea : News article classification
2. a/nlp-chat      (cosine score: 0.775)
   Idea : Chatbot using transformers for question answering
3. b/vision        (cosine score: 0.690)
   Idea : Object detection with deep learning
4. f/spam-detect   (cosine score: 0.656)
   Idea : Spam email classifier with machine learning
5. m/face-rec      (cosine score: 0.585)
   Idea : Face recognition attendance system
```

Demo run on a 16-repo sample dataset. Top 5 of 10 results shown.

Output: BM25

```
Enter your interests/skills: python, machine learning, nlp
Knowledge Graph matched topics : ['machine-learning', 'nlp', 'deep-learning', ...]
Knowledge Graph added topics   : ['transformers', 'computer-vision', 'data-science', ...]

TOP RECOMMENDED PROJECT TOPICS (BM25)
1. h/news-class    (bm25 score: 1.000)
   Idea : News article classification
2. a/nlp-chat      (bm25 score: 0.938)
   Idea : Chatbot using transformers for question answering
3. b/vision        (bm25 score: 0.693)
   Idea : Object detection with deep learning
4. i/recsys        (bm25 score: 0.592)
   Idea : Movie recommender system using collaborative filtering
5. m/face-rec      (bm25 score: 0.572)
   Idea : Face recognition attendance system
```

Demo run on a 16-repo sample dataset. Top 5 of 10 results shown.

Comparison: Rank of Each Repo

Repo	Cosine rank	BM25 rank	Change with BM25
h/news-class	1	1	same
a/nlp-chat	2	2	same
b/vision	3	3	same
f/spam-detect	4	9	lower (-5)
m/face-rec	5	5	same
i/recsys	6	4	higher (+2)
g/resume-parse	7	10	lower (-3)
p/data-viz	8	7	higher (+1)
o/qa-bert	9	8	higher (+1)
d/sentiment	10	6	higher (+4)

Highlighted rows = rank changed. Demo run on a 16-repo sample dataset.

Comparison Summary

What we observed

- Same top 3 results; 4 of the top 5 are shared
- BM25 moves f/spam-detect (4 to 9) and g/resume-parse (7 to 10) down
- BM25 moves i/recsys (6 to 4) and d/sentiment (10 to 6) up
- Precision@5 and @10 are equal (1.00 each)

Interpretation

- Results are mixed: neither method is clearly better on this sample
- BM25 rewards repos that match many distinct query words, including Knowledge Graph terms (our hypothesis)
- 16 repos and a loose relevance rule: no winner can be declared

Next: run both methods on the full GitHub dataset and compare Precision@k (compare_similarity.py).

Technologies & Design Choices

Python
Language

requests
GitHub API

pandas
Dataset handling

networkx
Knowledge Graph

scikit-learn
TF-IDF, cosine

- Content-based: works for a brand-new student (no cold start)
- TF-IDF + cosine: simple, fast, explainable, no training or GPU
- Knowledge Graph: goes beyond exact keyword matches

Limitations & Future Work

Limitations

- TF-IDF matches words, not meaning ("car" vs "automobile")
- Only recommends what is in the dataset
- Quality depends on how repos are tagged
- GitHub API rate limits slow the first download

Future work

- Semantic embeddings (Sentence-BERT)
- Curated ontology-based Knowledge Graph
- Web interface (Streamlit / Flask)
- Evaluation with precision@k and recall@k
- Filters: difficulty level, preferred language

Conclusion

A content-based recommender that uses TF-IDF and cosine similarity on real GitHub repositories, enhanced with a Knowledge Graph for semantic query expansion, to deliver ranked, personalized project topic suggestions.

Thank you. Questions?